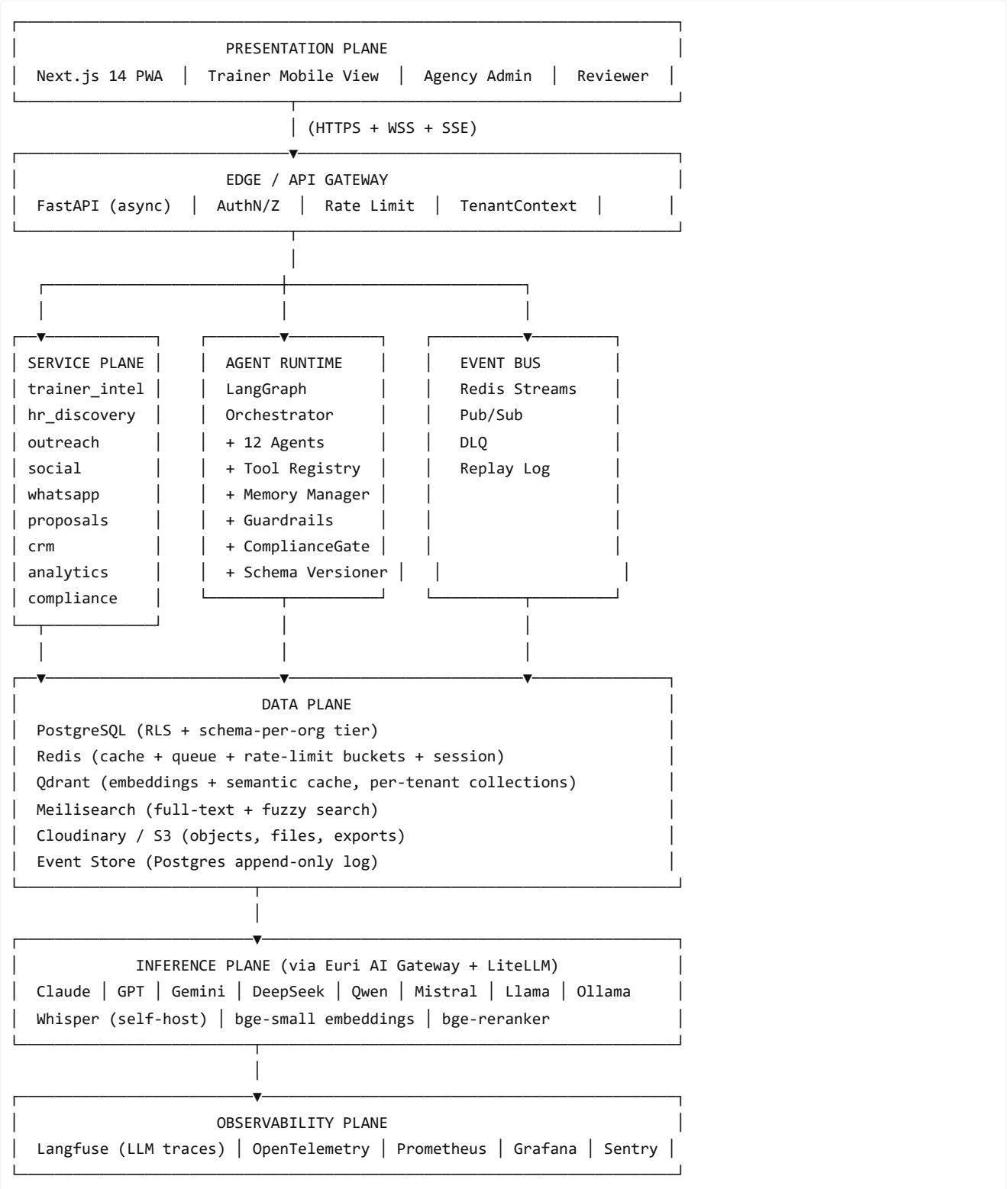
CorporateMind AI — Architecture Reference

This document is an engineer's map of the system wiring. For product context, see `PRD.md`. For detailed rules, see `CLAUDE.md` and `.claude/rules/*.md`.

---

1. Plane View



---

2. Monorepo Layout

```
CorporateMind AI/
├── apps/
│   ├── api/                          # Backend — FastAPI modular monolith
│   │   ├── src/corpmind/
│   │   │   ├── main.py               # FastAPI app factory
│   │   │   ├── core/
│   │   │   │   ├── config.py         # pydantic-settings
│   │   │   │   ├── db.py             # async SQLAlchemy
│   │   │   │   ├── tenancy.py        # TenantContext
│   │   │   │   ├── rbac.py           # Casbin policies
│   │   │   │   └── security.py       # auth, secrets
│   │   │   ├── ai/
│   │   │   │   ├── euri_client.py    # SOLE LLM egress
│   │   │   │   ├── prompts/          # versioned prompts
│   │   │   │   └── providers/        # fallback chains
│   │   │   ├── agents/               # LangGraph graphs + tools
│   │   │   │   ├── root_orchestrator.py
│   │   │   │   ├── trainer_profile/
│   │   │   │   ├── hr_discovery/
│   │   │   │   ├── outreach/
│   │   │   │   ├── proposals/
│   │   │   │   ├── compliance_guard/
│   │   │   │   └── ...
│   │   │   ├── modules/              # 7 Pillars
│   │   │   │   ├── trainer_intel/
│   │   │   │   ├── hr_discovery/
│   │   │   │   ├── outreach/
│   │   │   │   ├── social/
│   │   │   │   ├── whatsapp/
│   │   │   │   ├── proposals/
│   │   │   │   ├── crm/
│   │   │   │   ├── analytics/
│   │   │   │   ├── compliance/
│   │   │   │   └── ...
│   │   │   ├── channels/             # ChannelAdapter ABC + implementations
│   │   │   │   ├── base.py
│   │   │   │   ├── email_smtp.py
│   │   │   │   ├── whatsapp_cloud.py
│   │   │   │   └── ...
│   │   │   ├── workers/              # Celery tasks + beat schedule
│   │   │   │   ├── celery_app.py
│   │   │   │   ├── agents_tasks.py
│   │   │   │   ├── outreach_tasks.py
│   │   │   │   └── ...
│   │   │   └── ingestion/            # OCR, transcription, parsing
│   │   ├── tests/
│   │   │   ├── unit/
│   │   │   ├── repo/
│   │   │   ├── api/
│   │   │   ├── integration/
│   │   │   └── isolation/
│   │   ├── alembic/                  # Migrations (expand-then-contract)
│   │   └── pyproject.toml
```

```
│   │
│   └── web/                          # Frontend — Next.js 14 App Router
│       ├── app/
│       │   ├── (marketing)/          # Public pages
│       │   ├── (dashboard)/          # Tenant UI (protected)
│       │   ├── (admin)/              # Admin-only views
│       │   └── layout.tsx
│       ├── features/
│       │   ├── trainer_profile/      # Pilot feature
│       │   ├── hr_discovery/
│       │   ├── campaigns/
│       │   └── ...
│       ├── lib/
│       │   ├── api.ts                # API client wrapper
│       │   ├── auth.ts               # NextAuth config
│       │   └── hooks/
│       └── package.json
│
├── packages/
│   ├── shared-types/                 # Generated from OpenAPI
│   │   └── src/
│   │       ├── client.ts             # TS SDK (generated)
│   │       └── schemas.ts            # Zod + type exports
│   └── eslint-config/
│
├── infra/
│   ├── docker/
│   │   ├── compose.dev.yml           # Local dev stack (PG + Redis + Qdrant)
│   │   └── Dockerfile                # API / worker containers
│   ├── grafana/
│   │   └── dashboards/               # .json per domain
│   ├── scripts/
│   │   ├── deploy.sh
│   │   └── rollback.sh
│   └── terraform/                    # (Phase 2+, multi-region)
│
├── ops/
│   ├── runbooks/
│   │   ├── euri-provider-down.md
│   │   ├── compliance-guard-mass-block.md
│   │   └── ...
│   └── secrets.md                    # Secret owners + rotation
│
├── docs/
│   ├── PRD.md                        # Product spec (34 sections + 6 appendices)
│   ├── architecture.md               # This file
│   ├── adr/                          # Architecture Decision Records
│   │   ├── 0001-modular-monolith.md
│   │   ├── 0002-euri-as-sole-llm-egress.md
│   │   └── ...
│   └── exports/                      # CI-generated: OpenAPI, SDKs (gitignored)
│       └── README.md
│
├── .claude/
│   ├── settings.json
│   ├── scripts/
```

```
│   │   ├── post-edit-check.sh
│   │   └── block-direct-llm-imports.sh
│   ├── rules/                        # 22 domain rule files (loaded on-demand)
│   ├── skills/                       # 15 reusable workflow skills
│   └── MEMORY.md
│
├── .github/
│   └── workflows/                    # CI: lint, test, migrate, build, deploy
│
├── CLAUDE.md                         # Thin operating manual (auto-loaded)
├── README.md                         # Getting started
└── [other config: .gitignore, etc]
```

---

3. Module Anatomy (Ports & Adapters)

Every module under `apps/api/src/corpmind/modules/<name>/` follows this pattern:

```
<module>/
├── api.py           # HTTP endpoints (routes call service)
├── service.py       # Business logic (orchestrates repos)
├── repo.py          # Data access (SQLAlchemy queries + Qdrant)
├── models.py        # SQLAlchemy ORM models
├── schemas.py       # Pydantic v2 request/response schemas
└── events.py        # Domain events emitted
```

Inter-module rules:

- Modules NEVER import each other's `repo.py` or `models.py`.
- Cross-module communication happens via `service` interfaces (dependency injection) or the event bus.
- Example: `outreach` module wants to check a `trainer` profile. It calls `trainer_service.get_profile(trainer_id)` (DI'd), not `trainer_repo.find_by_id(trainer_id)` (violates isolation).

---

4. The 7 Pillars

Pillar	Module Path	What it does	Key Agents
Trainer Intelligence	`modules/trainer_intel`	Extract niche, topics, tone, pricing from uploads	TrainerProfileAgent
HR Discovery	`modules/hr_discovery`	Find matching companies + HR contacts	HRDiscoveryAgent
Outreach AI	`modules/outreach`	Generate personalized per-recipient messages	OutreachAgent
Social Automation	`modules/social`	Multi-channel content distribution	InstagramAgent, FacebookAgent, TelegramAgent, LinkedInAgent
WhatsApp Engine	`modules/whatsapp`	Official WA Business Cloud integration	WhatsAppAgent
Proposals	`modules/proposals`	AI pitch deck generation	ProposalAgent
CRM + Analytics	`modules/crm`, `modules/analytics`	Lead pipeline, campaign metrics	AnalyticsAgent, CampaignOptimizer

Multi-Agent Runtime	`agents/` + LangGraph	Orchestrator, tools, state, checkpoints	RootOrchestrator + 11 specialists

---

5. Agent Topology

12 specialized agents coordinating via shared `AgentState` (TypedDict):

Agent	Role	Tools	Model Tier	Memory Class
RootOrchestrator	Decomposes intent, manages global state, HITL escalator	All downstream agents + tools	Claude Sonnet/GPT-4.1	Working + Episodic
TrainerProfileAgent	Extracts niche, topics, tone, pricing, fit from uploads	OCR, Whisper, Vision, Qdrant write	Mid + Vision	Long-term Semantic
HRDiscoveryAgent	Finds matching companies + HR contacts from opt-in sources	Web search, Tavily, registries, Qdrant	Mid (reasoning) + Small (extraction)	Long-term Semantic
OutreachAgent	Drafts per-recipient messages with A/B variants	Trainer profile, HR record, Qdrant retrieval	Claude Sonnet (copy)	Working + Episodic
ProposalAgent	Generates pitch decks, agendas, pricing	Trainer profile, templates, PDF render	Claude Sonnet	Long-term Semantic
WhatsAppAgent	Template management, 24h window, follow-ups	WA Business Cloud API	Small + rule-based	Short-term Conversational
TelegramAgent	Channel broadcasts, community nurture	Telegram Bot API	Small	Short-term
InstagramAgent	Reel captions, story automation, hashtag intelligence	IG Graph API, image gen	Mid (creative)	Episodic
FacebookAgent	Page publishing, event promotion, messenger	FB Graph API	Mid	Episodic
LinkedInAgent	Public company-page posts, public-data lookups (no DMs)	LI public APIs	Mid	Episodic
CampaignOptimizer	Post-hoc analysis: which segment × channel × tone wins	Analytics DB, Langfuse	Small + analytical	Stateless
AnalyticsAgent	Daily rollups, insight cards, anomaly detection	Postgres analytics, time-series	Small	Stateless
ComplianceGuardAgent	Gate every outbound message: opt-in, rate, dedup, content	Opt-in DB, rate buckets, classifier	Small + rules	Stateless
LLMOpsGuardian	Monitors quality, triggers retries, escalates to HITL	Promptfoo evals, Langfuse, validators	Small + rule-based	Stateless

Agent communication: All agents talk via the shared `AgentState` (never directly) — this keeps the topology a tree (deterministic, debuggable), not a mesh.

---

6. Data Stores

Store	Role	Tenancy Model	Key Tables/Collections
PostgreSQL 16	Primary OLTP, event log, state	RLS on every base table; `tenant_id` composite index	orgs, workspaces, users, trainers, companies, hr_contacts, campaigns, outreach_messages, agent_runs, workflow_checkpoints, events
Redis	Cache, queue broker, rate-limit buckets, session, pubsub	Key prefix `t:{org_id}:{ws_id}:...` with per-tenant memory cap	Celery queues, cache, rate buckets, conversation sessions, flag cache
Qdrant	Embeddings, semantic cache, retrieval	Per-tenant collections: `trainer_profiles_{org_id}`, `companies_{org_id}`, etc. Global `prompt_cache_global` (PII-scrubbed)	Vectors 768-d (bge-small), payloads with tenant_id predicates
Meilisearch	Full-text + fuzzy search	Per-tenant indices	company names, hr contact names, campaign summaries
Cloudinary / S3	Objects, PDFs, exports	Paths under `tenants/{org_id}/workspaces/{ws_id}/...` with signed URLs	Uploaded posters, generated proposals, exported reports

---

7. Key Operational Flows

7a. Outbound Message Gate → Send

```
Agent generates message outline
    ↓
OutreachAgent drafts copy per recipient
    ↓
Message queued for send (campaign_recipients row)
    ↓
ComplianceGuardAgent checks:
  • opt-in(contact, channel)?
  • unsubscribe list?
  • frequency cap (≤2/week)?
  • WA 24h window?
  • duplicate?
  • content policy?
  • tenant budget?
    ↓ PASS: queue send task
    ↓ BLOCK: log reason + notify trainer
    ↓ HITL: pause for approval
    ↓
Rate-limited send (per-channel token bucket)
    ↓
Delivery tracked (sent_at, delivery status, open, click, reply)
    ↓
Outcome event emitted → analytics
```

7b. Agent Run Lifecycle

```
HTTP request → RootOrchestrator (start)
    ↓ Hydrate TenantContext, load memory
    ↓
Plan: LLM generates structured task list
    ↓
Execute: Dispatcher runs each task deterministically
    │  (tool call → validation → side effect)
    │
    └─ on tool failure: retry (exponential backoff)
       on schema failure: re-prompt (max 2)
       on low confidence: HITL
    ↓
Verify: validators check outputs grounded in facts
    ↓
Checkpoint: persist state to workflow_checkpoints
    ↓
Langfuse span emitted (tokens, cost, latency)
    ↓
Outcome event → RLHF feedback store
```

7c. HITL Pause / Resume

```
Agent route → HITL gate triggered (recipients > 200, etc)
    ↓
State checkpointed to Postgres (workflow_checkpoints)
    ↓
Campaign status = "pending_approval"
    ↓
Trainer sees approval card in dashboard
    ↓
Trainer approves / edits / rejects
    ↓
On approve: resume_workflow event → async task loads checkpoint
    ↓
LangGraph resumes from last node, continues
    ↓
Outcome emitted
```

---

8. Inference Plane: Euri Routing

Every LLM call routes through `EuriClient` (singleton) to `euri_client.py`:

```
Call( task="outreach_copy", prompt_name="outreach.email.v3", ... )
    ↓
PromptInjectionFilter (scrub user input)
    ↓
PIIRedactor (Presidio + regex)
    ↓
SemanticCache (Qdrant prompt_cache_global, cosine ≥ 0.96)
    ↓ MISS: continue
    ↓
Routing matrix: task → (primary, fallback_chain)
```

```
    Routing decision: tenant_plan, budget_remaining, latency_target
    ↓
Primary model (e.g., Claude Sonnet)
    ↓ success: continue
    ↓ timeout/error: fallback chain
       Secondary (e.g., GPT-4.1)
       Tertiary (e.g., Gemini)
       Local (e.g., Llama 3.3 on Ollama)
    ↓
OutputModerator (Llama Guard 3, schema validation)
    ↓
Langfuse span: tenant, agent, tokens, cost, model, cached
    ↓
Return to caller
```

Model selection policy (default):

- Cheap (DeepSeek, Qwen, Gemini Flash, Haiku): classification, extraction, ranking, dedupe.
- Premium (Claude Sonnet/Opus, GPT-4-class): personalized outreach, proposals, strategic copy.

---

9. Operational Layers: CI/CD + Environments

Pipeline

```
PR opened
  │ → ruff + mypy + eslint + tsc (lint)
  │ → pytest (unit + repo + api)
  │ → alembic upgrade (check migrations)
  │ → OpenAPI diff (no breaking changes)
  │ → build Docker (multi-stage)
  │ → (optional) Promptfoo eval (if prompts touched)
  │ → (optional) tenant-isolation regression (if new table)
  ↓
PR merged to main
  │ → all CI gates pass
  │ → deploy preview (Vercel web + Railway api)
  ↓
main branch tagged + released
  │ → deploy staging (full staging env refresh)
  │ → canary 10% prod (blue/green, 15 min soak)
  │ → monitor SLOs: error rate, p95 latency, LLM fallback
  ↓
Canary green: proceed to 100% rollout
Canary red: auto-rollback to previous
```

Environments

Env	Composed of	Data	Deploy trigger
dev	Local Docker Compose (PG, Redis, Qdrant, Mailhog)	Fixtures	On `make dev`
preview	Vercel + Railway per PR	PR-scoped test data	PR → deployment URL
staging	Full prod replica topology	Daily snapshot restore from prod	`main` branch merge
prod	Vercel + Railway + Cloudflare CDN	Live tenant data	Canary approval

---

10. Architecture Decision Records (ADR Index)

ADR	Title	Status	Context
ADR-0001	Modular Monolith for Stage 1	Accepted	Balances simplicity (no K8s) with scalability hooks for Stage 2+
ADR-0002	Euri as Sole LLM Egress	Accepted	Cost telemetry, fallback chains, vendor lock-in mitigation
ADR-0003	Postgres RLS as Tenant Isolation Default	Accepted	Strong isolation + cost-efficient until > 250 trainers/org
ADR-0004	LangGraph for Agent Orchestration	Accepted	Native checkpointing, deterministic replay, typed state
ADR-0005	Celery for Async Task Distribution	Accepted	Proven, per-tenant queue caps, low operational overhead
ADR-0006	Expand-Then-Contract Migrations	Accepted	Zero-downtime deploys, safe rollback, reversibility

See `docs/adr/` for full ADR library.

---

11. Stage Evolution: Scaling Triggers

Stage 1 → Stage 2 Trigger

Move to multi-service extraction when **ANY** of:

- API p95 latency > 800ms sustained
- Celery queue depth > 200 sustained
- Database connections approaching pool limit

Stage 2 changes: Extract `social/` and `whatsapp/` modules as separate services. Postgres read-replica. Redis clustered. Qdrant paid tier.

Stage 2 → Stage 3 Trigger

Move to distributed (Kubernetes) when **ANY** of:

- ARR > ₹4 Cr
- Tenant count > 5,000
- First Enterprise dedicated-VPC request

Stage 3 changes: Kubernetes (Hetzner/EKS). Postgres sharded (Citus or per-region). Kafka for event bus. GPU pool for Llama 70B.

Quick Navigation

- **Rules?** → `CLAUDE.md` (thin master) + `.claude/rules/` (22 domain files)
- **Getting started?** → `README.md` + `PRD.md` Section 31 (Phase 0 roadmap)
- **Adding a new agent?** → Need an ADR + `/create-langgraph-agent` skill
- **Adding a new channel?** → `/create-channel-adapter` skill
- **Adding a new module?** → `/create-module` skill
- **Pre-PR checklist?** → `CLAUDE.md` (Pre-PR Checklist section)
- **Incident?** → `.claude/rules/incident-response.md` + `ops/runbooks/`
- **Observability?** → `.claude/rules/observability.md` + Grafana dashboards in `infra/grafana/`

Last updated: 2026-05-24 | For questions, open an issue or check `CLAUDE.md` .